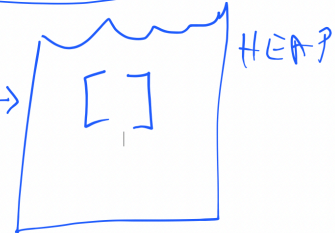


useReducer

```
export default function taskReducer(state, action) {  
  switch(action.type) {  
    case "ADD_TASK":  
      // clone existing task states and add new task  
      return [...state, { id: Date.now(), text: action.payload, completed: false}]  
    case "TOGGLE_TASK":  
      return state.map(task => task.id === action.payload ?  
        {...task, completed: ! task.completed}: task)  
    case "REMOVE_TASK":  
      return state.filter(task => task.id !== action.payload)  
    default:  
      return state;  
  }  
}
```

`const initialState = [];`

```
export default function TaskManager() {  
  let [tasks, dispatch] = useReducer(taskReducer, initialState);  
}
```



```
export default function taskReducer(state, action) {  
  switch(action.type) {  
    case "ADD_TASK":  
      // clone existing task states and add new task  
      return [...state, { id: Date.now(), text: action.payload, completed: false}]  
    case "TOGGLE_TASK":  
      return state.map(task => task.id === action.payload ?  
        {...task, completed: ! task.completed}: task)  
    case "REMOVE_TASK":  
      return state.filter(task => task.id !== action.payload)  
    default:  
      return state;  
  }  
}
```

`const initialState = [];`

```
export default function TaskManager() {  
  let [tasks, dispatch] = useReducer(taskReducer, initialState);  
}
```

delegate/dispatch action

2B

2A

3



```
<button type="button"  
  onClick={() => dispatch({'type': 'ADD_TASK', payload:taskData})}>Add Task</button>
```

①